Aula 13 — Registros e estruturas compostas

Representando pessoas, produtos e cadastros com dicionários em Python

Dieimes Nunes

Retomando a Aula 12

Na aula anterior, usamos matrizes para organizar dados em linhas e colunas.

Exemplo:

```
notas = [
    [8.0, 7.0, 9.0],
    [6.0, 8.0, 7.0],
    [9.0, 9.0, 10.0]
]
```

Matrizes são muito úteis quando os dados possuem uma estrutura tabular semelhante.

Mas hoje vamos enfrentar outro tipo de problema.

O problema da aula

Precisamos representar um aluno com:

```
Nome: Ana Paula
Idade: 17
Matrícula: 2026001
Média: 8.5
Ativo: Sim
```

Essas informações possuem tipos e significados diferentes.

Poderíamos usar várias variáveis:

```python
nome = "Ana Paula"
idade = 17
matricula = "2026001"
media = 8.5
ativo = True
```

Mas como podemos dizer que todos esses dados pertencem ao **mesmo aluno**?

O que é um registro?

Um **registro** reúne diferentes informações relacionadas a uma mesma entidade.

Exemplos de entidades:

- aluno;
- cliente;
- funcionário;
- produto;
- livro;
- veículo.

Um registro de produto pode ter:

Código
Nome
Preço
Quantidade
Categoria

Cada informação é um **campo** do registro.

Atividade 1 — Identificando registros e campos

No caderno, escolha três entidades:

1. aluno;
2. produto;
3. veículo.

Para cada uma, escreva pelo menos cinco campos.

Exemplo de modelo:

```
Entidade: _____________________
Campo 1: _____________________
Campo 2: _____________________
Campo 3: _____________________
```

```
Campo 4: _____________________
Campo 5: _____________________
```

Depois indique o tipo de dado esperado para cada campo.

Dicionários em Python

Em Python, uma forma adequada de representar registros é o **dicionário**.

Exemplo:

```
aluno = {
    "nome": "Ana Paula",
    "idade": 17,
    "matricula": "2026001",
    "media": 8.5,
    "ativo": True
}
```

O dicionário guarda pares:

`chave → valor`

Chave e valor

No registro:

```
aluno = {
    "nome": "Ana Paula",
    "idade": 17,
    "media": 8.5
}
```

Temos:

chave	valor
"nome"	"Ana Paula"
"idade"	17
"media"	8.5

A chave identifica o campo.

O valor guarda a informação.

Acessando campos

Podemos acessar um valor pela chave:

```
print(aluno["nome"])
print(aluno["idade"])
print(aluno["media"])
```

Resultado:

```
Ana Paula
17
8.5
```

Aqui não usamos um índice numérico.

Usamos o nome do campo.

Atividade 2 — Primeiro registro

Crie:

`registro_produto.py`

Crie um dicionário chamado `produto` com:

- código;
- nome;
- preço;
- quantidade;
- categoria.

Depois mostre cada campo em uma linha.

Exemplo de saída:

```
Código: 101
Produto: Teclado
Preço: R$ 120.00
Quantidade: 5
Categoria: Periféricos
```

Alterando um campo

Podemos atribuir um novo valor:

```
produto["preco"] = 130.00
produto["quantidade"] = 8
```

Depois:

```
print(produto)
```

O registro continua sendo o mesmo, mas os campos foram atualizados.

Adicionando um novo campo

Também podemos incluir uma nova chave:

```
produto["ativo"] = True
```

Agora o registro possui mais uma informação.

Isso mostra que dicionários são estruturas flexíveis.

Removendo um campo

Podemos usar `del`:

```
del produto["categoria"]
```

Ou `pop()`:

```
produto.pop("categoria")
```

Antes de remover dados, precisamos entender se o campo é realmente dispensável para o sistema.

Atividade 3 — Atualizando um cadastro

Use o registro de produto da atividade anterior.

Faça o programa:

1. mostrar o produto original;
2. aumentar a quantidade em estoque em 10 unidades;
3. alterar o preço;
4. adicionar o campo `ativo`;
5. mostrar o registro atualizado.

Depois responda:

Lendo dados para um registro

Podemos construir o registro com dados digitados:

```
aluno = {}

aluno["nome"] = input("Nome: ")
aluno["idade"] = int(input("Idade: "))
aluno["matricula"] = input("Matrícula: ")
aluno["media"] = float(input("Média: "))
```

Depois:

```
print(aluno)
```

O dicionário começa vazio e recebe os campos.

Outra forma de criar o registro

Podemos primeiro receber os dados:

```
nome = input("Nome: ")
idade = int(input("Idade: "))
media = float(input("Média: "))
```

E depois montar:

```
aluno = {
    "nome": nome,
    "idade": idade,
    "media": media
}
```

As duas formas são válidas.

A escolha depende da organização do programa.

Atividade 4 — Cadastro de aluno

Crie:

`cadastro_aluno.py`

Solicite:

- nome;
- idade;
- matrícula;
- nota 1;
- nota 2.

Calcule a média e guarde também no registro.

O registro deve terminar parecido com:

```
{
    "nome": "Ana",
    "idade": 17,
    "matricula": "123",
    "nota1": 8.0,
    "nota2": 7.0,
    "media": 7.5
}
```

Mostre uma ficha organizada na tela.

Um registro é suficiente?

Até agora temos um aluno:

```
aluno = {
    "nome": "Ana",
    "media": 8.0
}
```

Mas um sistema normalmente possui vários alunos.

Podemos combinar estruturas já estudadas:

> **uma lista de dicionários.**

Lista de registros

Exemplo:

```
alunos = [
    {"nome": "Ana", "media": 8.0},
    {"nome": "Bruno", "media": 6.5},
    {"nome": "Carla", "media": 9.0}
]
```

Aqui:

- `alunos` é uma lista;
- cada elemento da lista é um registro;
- cada registro possui campos.

Isso representa uma pequena tabela de dados.

Acessando lista + registro

```
print(alunos[0])
```

Mostra o primeiro registro.

```
print(alunos[0]["nome"])
```

Mostra:

```
Ana
```

Podemos pensar assim:

```
alunos[0]          → primeiro aluno
["nome"]          → campo nome desse aluno
```

Atividade 5 — Lendo uma tabela de registros

Considere:

```
produtos = [
    {"codigo": 1, "nome": "Mouse", "preco": 80.0},
    {"codigo": 2, "nome": "Teclado", "preco": 120.0},
    {"codigo": 3, "nome": "Monitor", "preco": 900.0}
]
```

Sem executar, responda:

```
produtos[1]["nome"] = ?
produtos[2]["preco"] = ?
produtos[0]["codigo"] = ?
```

Depois escreva o comando para mostrar o preço do teclado.

Percorrendo registros

Podemos usar `for`:

```
for aluno in alunos:
    print(aluno["nome"], aluno["media"])
```

A variável `aluno` recebe um dicionário por vez.

Isso deixa o código mais claro do que manter várias listas relacionadas.

Comparando com duas listas

Antes poderíamos fazer:

```
nomes = ["Ana", "Bruno", "Carla"]
medias = [8.0, 6.5, 9.0]
```

Era necessário manter a mesma posição.

Com registros:

```
alunos = [
    {"nome": "Ana", "media": 8.0},
    {"nome": "Bruno", "media": 6.5},
    {"nome": "Carla", "media": 9.0}
]
```

Nome e média permanecem juntos no mesmo registro.

Atividade 6 — Listando alunos

Crie uma lista com quatro alunos.

Cada aluno deve possuir:

- matrícula;
- nome;
- média;
- frequência.

Percorra a lista e mostre:

```
Matrícula: ...
Nome: ...
Média: ...
```

```
Frequência: ...
----------------------
```

Depois acrescente a situação:

```
Aprovado / Reprovado
```

usando as regras já estudadas.

Cadastrando vários registros

Podemos criar um registro dentro da repetição:

```
alunos = []

for i in range(3):
    aluno = {}

    aluno["nome"] = input("Nome: ")
    aluno["media"] = float(input("Média: "))

    alunos.append(aluno)
```

Importante:

> Um novo dicionário é criado a cada repetição.

Atividade 7 — Cadastro de vários produtos

Crie:

`cadastro_produtos.py`

Pergunte quantos produtos serão cadastrados.

Cada produto deve possuir:

- código;
- nome;
- preço;
- quantidade.

Armazene todos em uma lista de dicionários.

Depois mostre os registros cadastrados.

Valide preço e quantidade para não aceitar valores negativos.

Pesquisa em registros

Podemos pesquisar por um campo.

Exemplo por matrícula:

```
matricula_busca = input("Matrícula: ")
encontrado = False

for aluno in alunos:
    if aluno["matricula"] == matricula_busca:
        print(aluno)
        encontrado = True
        break

if not encontrado:
    print("Aluno não encontrado")
```

Isso é uma pesquisa sequencial aplicada a registros.

Atividade 8 — Buscar produto pelo código

Utilize sua lista de produtos.

Solicite um código ao usuário.

Se encontrar, mostre:

- nome;
- preço;
- quantidade;
- valor total em estoque.

Calcule:

```
valor_estoque = preço × quantidade
```

Se não encontrar, mostre uma mensagem adequada.

Alterando um registro encontrado

Podemos modificar o próprio registro durante a pesquisa.

```
codigo = int(input("Código: "))

for produto in produtos:
    if produto["codigo"] == codigo:
        produto["preco"] = float(input("Novo preço: "))
        print("Produto atualizado")
        break
```

Como `produto` representa o dicionário da lista, a alteração permanece na estrutura.

Atividade 9 — Atualização de estoque

Crie uma opção para:

1. buscar um produto pelo código;
2. informar quantidade de entrada;
3. somar essa quantidade ao estoque atual;
4. mostrar o estoque atualizado.

Exemplo:

```
Estoque atual: 5
Entrada: 8
Novo estoque: 13
```

Mostrando registros em formato de tabela

Podemos organizar a saída:

```
print(f"{'CÓDIGO':<8} {'NOME':<20} {'PREÇO':>10}")

for produto in produtos:
    print(
        f"{produto['codigo']:<8} "
        f"{produto['nome']:<20} "
        f"{produto['preco']:>10.2f}"
    )
```

A formatação melhora a leitura do relatório.

Atividade 10 — Relatório tabular

Com a lista de produtos, mostre uma tabela contendo:

```
CÓDIGO | PRODUTO | PREÇO | QTD | TOTAL
```

O total deve ser:

```
preço × quantidade
```

Ao final, mostre:

- quantidade de produtos cadastrados;
- soma do valor de todo o estoque;
- produto com maior preço;
- produto com menor preço.

Dicionário dentro de dicionário

Também podemos ter estruturas mais complexas.

Exemplo:

```
aluno = {
    "nome": "Ana",
    "contato": {
        "email": "ana@email.com",
        "telefone": "99999-9999"
    }
}
```

Para acessar:

```
print(aluno["contato"]["email"])
```

Esse tipo de estrutura aparece em muitos sistemas reais.

Não precisamos exagerar na complexidade neste momento.

Dicionário com lista

Um campo também pode conter uma lista:

```
aluno = {
    "nome": "Ana",
    "notas": [8.0, 7.5, 9.0]
}
```

Podemos calcular:

```
soma = 0

for nota in aluno["notas"]:
    soma += nota

media = soma / len(aluno["notas"])
```

Estamos combinando vários conteúdos estudados.

Atividade 11 — Registro com lista de notas

Crie um registro de aluno com:

- matrícula;
- nome;
- lista de três notas.

O programa deve:

1. cadastrar as notas;
2. calcular a média;
3. guardar a média no próprio registro;
4. definir a situação;
5. mostrar a ficha completa.

Depois transforme para cadastrar três alunos em uma lista.

E os arquivos indexados da ementa?

A ideia de **índice** aparece quando queremos localizar registros por uma chave, como:

- matrícula;
- CPF;
- código de produto;
- identificador único.

Neste curso, primeiro estamos construindo a lógica com estruturas em memória.

Mais adiante veremos como gravar registros em arquivos.

Chave de identificação

Em um cadastro, determinados campos precisam identificar o registro.

Exemplo:

```
Produto → código
Aluno   → matrícula
Cliente → CPF ou código
Livro   → ISBN ou código interno
```

Um bom sistema deve evitar registros duplicados quando a chave precisa ser única.

Atividade 12 — Evitando código duplicado

Ao cadastrar um novo produto:

1. solicite o código;
2. percorra a lista atual;
3. verifique se o código já existe;
4. se existir, não cadastre;
5. se não existir, continue o cadastro.

Pense antes de programar:

Em qual momento o novo dicionário deve ser adicionado à lista?

Erros comuns nesta aula

Chave inexistente

```
print(aluno["telefone"])
```

Se a chave não existe, ocorre erro.

Confundir lista e dicionário

```
alunos["nome"]
```

se `alunos` for uma lista.

Primeiro precisamos escolher um elemento:

```
alunos[0]["nome"]
```

Reaproveitar o mesmo registro incorretamente

Precisamos criar um novo dicionário para cada cadastro.

Onde estamos no curso

Agora temos estruturas mais poderosas:

```
Variável              → um valor
Lista / vetor          → vários valores
Matriz                 → linhas e colunas
Dicionário / registro → campos de uma entidade
Lista de dicionários  → vários registros
```

Nosso código, porém, está começando a ficar grande.

Na próxima aula vamos aprender a dividir o programa em partes menores usando **funções**.

Desafio final da aula

Crie:

```
sistema_estoque_registros.py
```

Menu:

```
1 - Cadastrar produto
2 - Listar produtos
3 - Buscar produto
4 - Alterar preço
5 - Entrada de estoque
6 - Saída de estoque
7 - Relatório geral
0 - Sair
```

Cada produto deve ser um dicionário com:

```
código
nome
preço
```

```
quantidade
categoria
```

Regras:

- código não pode se repetir;
- preço não pode ser negativo;
- estoque não pode ficar negativo;
- pesquisa deve usar o código;
- relatório deve mostrar o valor total de cada produto.

Fechamento

Nesta aula, trabalhamos com:

- conceito de registro;
- campos;
- dicionários;
- chave e valor;
- alteração de registros;
- lista de dicionários;
- cadastro de vários registros;
- pesquisa por campo;
- atualização de registros;
- relatórios tabulares;
- campo-chave e prevenção de duplicidade.